Git & GitHub Fundamentals

Day 4 Notes · DevOps Sprint 9 · Industry Workflow Basics

---

1. What is Version Control?

Imagine you're writing a document and you keep saving new copies: essay_final, essay_final2, essay_actually_final. That's basically what version control replaces, except it does it properly. It's a system that tracks every change made to your files over time, who made the change, and why, so you never lose work and never have to guess which file is the real final version.

In coding terms, version control lets a developer save a snapshot of their project at any point, go back to an older snapshot if something breaks, and see exactly what changed between two points in time. Git is the tool that does this job, and it's by far the most widely used one in the industry today.

2. Why Do Software Companies Use Git?

Software is rarely built by one person working alone. A typical product has multiple developers touching the same codebase, sometimes the same files, on the same day. Without a system to manage that, chaos is guaranteed — people would overwrite each other's work, nobody would know which version is live, and bugs would be nearly impossible to trace back to their source.

Git solves this by letting every developer work independently, then combine their changes safely. Companies rely on it because it gives them a complete history of the project, makes teamwork possible without stepping on each other's toes, and provides a safety net — if something goes wrong in production, you can always trace it back and even undo it.

3. What Problems Does Git Solve?

- **Losing work:** Every meaningful change is saved as a snapshot, so nothing is ever truly lost.
- **Conflicting edits:** Multiple people can work on the same project at the same time without directly overwriting each other.
- **No accountability:** Every change is tied to a specific person and comes with a message explaining why it was made.
- **Fear of breaking things:** Developers can experiment freely in isolated branches without risking the stable, working version of the project.
- **No way to roll back:** If a bug slips into the code, Git makes it possible to see exactly what changed and revert it.

4. Difference Between Git and GitHub

This is probably the most commonly confused pair of terms for beginners, so it's worth being precise about it.

Git is the actual version control tool. It runs locally on your computer, in the background, tracking changes to your project. You don't need the internet to use Git — it works entirely offline.

GitHub is a website (and company) that hosts Git repositories online. It's built around Git, but it's not Git itself. GitHub adds a layer on top: a place to store your code in the cloud, collaborate with other people, review each other's changes, and manage projects.

A simple way to remember it: Git is the engine, GitHub is one of many garages where you can park and share the car. Other companies like GitLab and Bitbucket offer similar hosting, all built on top of Git.

5. What is a Repository?

A repository, or "repo" for short, is simply the project folder that Git is keeping track of. It contains all your files plus a hidden record of every change ever made to them. A repository can live only on your computer (local repo) or be hosted online on a platform like GitHub (remote repo), and usually a project has both, kept in sync with each other.

6. What is a Commit?

A commit is a saved snapshot of your project at a specific point in time. Think of it as a checkpoint in a video game — if things go wrong later, you can always come back to that point. Every commit comes with a short message describing what was changed, which is what makes a project's history readable and useful months or years later.

Commits build on top of each other, forming a timeline. That timeline is essentially the entire story of how a project was built, one small change at a time.

7. What is a Branch?

A branch is an independent line of work that splits off from the main project. The main, stable version of a project usually lives on a branch called main (or sometimes master). Whenever someone wants to add a new feature or fix a bug, they create a new branch off of main, work on it in isolation, and only bring it back once it's ready.

This is what allows teams to work on several features at once without interfering with each other, and it keeps the main branch stable and deployable at all times.

8. What is a Merge?

Merging is the process of combining the changes from one branch into another — most commonly, taking a finished feature branch and folding it back into main. Git tries to combine the changes automatically. If the same lines of code were changed in two different ways on both branches, Git flags it as a "merge conflict," and a developer has to step in and decide which version to keep.

9. What is a Pull Request?

A pull request, or PR, is a GitHub feature (not a Git feature) used to propose merging one branch into another. Instead of a developer merging their own code straight into main, they open a pull request asking teammates to review it first.

This is where a lot of real collaboration happens: teammates comment on the code, suggest changes, run automated tests against it, and only approve the merge once everyone is satisfied. It's a core part of how professional teams maintain code quality and catch mistakes before they reach production.

Key Takeaway

Git and GitHub matter in DevOps because modern software is built by teams, shipped constantly, and needs to stay reliable while changing fast. Git gives every change a history and every developer a safe space to experiment; GitHub adds the collaboration layer on top so teams can review, discuss, and ship code together with confidence. That combination is exactly why almost every DevOps pipeline starts with a Git repository.